

Week 2: Category and ArtProduct Management System

Scenario

Arthub is an online art marketplace where different types of artworks are organized into categories. The database needs to store category details and artwork details such as artwork name, artist ID, category ID, price, and stock.

The purpose of this week is to create the Category and ArtProduct tables, establish relationships between them, apply constraints, insert records, perform CRUD operations, and generate category-wise reports.

1. Create Category and ArtProduct Tables

Category Table

The CREATE TABLE command is used to create a new table in the database. The Category table stores the details of different artwork categories. The table is initially created with the required columns. Constraints are added separately after creating the table.

```
SQL> create table category(category_ID varchar(10),category_Name varchar(30),Description varchar(200));
Table created.
```

ArtProduct Table

The ArtProduct table stores the details of artworks available in Arthub. The CREATE TABLE command creates the ArtProduct table with columns for artwork ID, artist ID, category ID, artwork name, price, and stock.

```
SQL> create table artproduct(artwork_ID varchar(10),artist_ID varchar(10),category_ID varchar(10),artwork_name varchar(100),price number(10,2),stock int);
Table created.
```

2. Describe the Table Structure

The DESCRIBE command is used to view the structure of a table. It displays the column names, data types, and other structural information. This command helps to verify whether the tables have been created with the required columns and data types.

```
SQL> desc category;
Name                                     Null?      Type
-----
CATEGORY_ID                             VARCHA2(10)
CATEGORY_NAME                           VARCHA2(30)
DESCRIPTION                             VARCHA2(200)
```

```
SQL> desc artproduct;
Name                                     Null?      Type
-----
ARTWORK_ID                             VARCHA2(10)
ARTIST_ID                              VARCHA2(10)
CATEGORY_ID                             VARCHA2(10)
ARTWORK_NAME                           VARCHA2(100)
PRICE                                  NUMBER(10,2)
STOCK                                  NUMBER(38)
```

3. Define Relationships and Constraints

A category can contain many artworks. Therefore, the relationship between Category and ArtProduct is **One-to-Many (1 → *)**.

Constraints are used to control the type of data that can be stored in the tables.

Primary Key Constraint

A Primary Key uniquely identifies each record in a table.

For the Category table, Category_ID is made the Primary Key using the ALTER TABLE command.

For the ArtProduct table, Artwork_ID is made the Primary Key.

```
SQL> alter table category add constraint pk_category primary key(category_ID);
Table altered.

SQL>
SQL> alter table artproduct add constraint pk_artwork primary key(artwork_ID);
Table altered.
```

Foreign Key Constraint

A Foreign Key is used to establish a relationship between two tables.

Category_ID in ArtProduct refers to Category_ID in Category.

This ensures that the category assigned to an artwork must exist in the Category table.

```
SQL> ALTER TABLE artproduct ADD CONSTRAINT fk_artwork_category FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID);
Table altered.
```

NOT NULL Constraint

The NOT NULL constraint prevents a column from containing an empty value.

For the Category table:

```
ALTER TABLE Category MODIFY Category_Name VARCHAR2(50) NOT NULL;
```

For the ArtProduct table:

```
ALTER TABLE ArtProduct MODIFY Artwork_Name VARCHAR2(100) NOT NULL;
```

Therefore, every category and artwork must have a name.

```
SQL> ALTER TABLE category MODIFY category_Name VARCHAR(30) NOT NULL;
Table altered.

SQL> ALTER TABLE artproduct MODIFY artwork_name VARCHAR(100) NOT NULL;
Table altered.
```

UNIQUE Constraint

The UNIQUE constraint prevents duplicate values in a column.

For Category:

```
ALTER TABLE Category ADD CONSTRAINT uq_category_name UNIQUE (Category_Name);
```

For ArtProduct:

```
ALTER TABLE ArtProduct ADD CONSTRAINT uq_artwork_name UNIQUE (Artwork_Name);
```

This prevents two categories or two artworks from having the same name.

```
SQL> ALTER TABLE Category ADD CONSTRAINT uq_category_name UNIQUE (category_Name);
Table altered.

SQL> ALTER TABLE artproduct ADD CONSTRAINT uq_artwork_name UNIQUE (Artwork_Name);
Table altered.
```

CHECK Constraint

The CHECK constraint ensures that the values entered satisfy a specified condition.

For Category ID:

```
ALTER TABLE Category ADD CONSTRAINT chk_category_id CHECK (Category_ID > 0);
```

For artwork price:

```
ALTER TABLE ArtProduct ADD CONSTRAINT chk_artwork_price CHECK (Price > 0);
```

For artwork stock:

```
ALTER TABLE ArtProduct ADD CONSTRAINT chk_artwork_stock CHECK (Stock >= 0);
```

These conditions ensure that IDs are positive, artwork prices are greater than zero, and stock cannot be negative.

```
SQL> ALTER TABLE Category ADD CONSTRAINT chk_category_id CHECK (Category_ID > 0);
Table altered.

SQL> ALTER TABLE artproduct ADD CONSTRAINT chk_artwork_price CHECK (Price > 0);
Table altered.
```

4. Insert Values

The INSERT INTO command is used to add records into a table.

Category records are inserted using the following command:

```
INSERT INTO Category VALUES (1, 'Paintings', 'Traditional and modern painted artworks');
```

Similarly, artwork records are inserted into ArtProduct:

```
INSERT INTO ArtProduct VALUES (101, 1, 1, 'Sunset Over Hills', 4500, 3);
```

Additional records can be inserted using the same INSERT INTO command.

The stored records can be displayed using:

```
SELECT * FROM Category;
```

```
SELECT * FROM ArtProduct;
```

```
SQL> INSERT INTO Category VALUES (1, 'Paintings', 'Traditional and modern painted artworks');
1 row created.

SQL> INSERT INTO Category VALUES (2, 'Sketches', 'Pencil and charcoal sketch artworks');
1 row created.

SQL> INSERT INTO Category VALUES (3, 'Digital Art', 'Artwork created using digital tools');
1 row created.

SQL> INSERT INTO Category VALUES (4, 'Photography', 'Creative and artistic photographs');
1 row created.

SQL> INSERT INTO Category VALUES (5, 'Sculptures', 'Three dimensional artistic creations');
1 row created.

SQL> INSERT INTO Category VALUES (6, 'Calligraphy', 'Decorative and artistic handwriting');
1 row created.

SQL> INSERT INTO Category VALUES (7, 'Prints', 'Printed reproductions of artworks');
1 row created.

SQL> INSERT INTO Category VALUES (8, 'Illustrations', 'Creative drawings and visual illustrations');
1 row created.

SQL> INSERT INTO Category VALUES (9, 'Abstract Art', 'Artworks using shapes colors and forms');
1 row created.

SQL> INSERT INTO Category VALUES (10, 'Craft Art', 'Handmade decorative artistic creations');
1 row created.
```

```
SQL> select* from category;
```

```
CATEGORY_I CATEGORY_NAME
-----
DESCRIPTION
-----
```

```
1          Paintings
Traditional and modern painted artworks
```

```
2          Sketches
Pencil and charcoal sketch artworks
```

```
3          Digital Art
Artwork created using digital tools
```

```
CATEGORY_I CATEGORY_NAME
-----
DESCRIPTION
-----
```

```
4          Photography
Creative and artistic photographs
```

```
5          Sculptures
Three dimensional artistic creations
```

```
6          Calligraphy
Decorative and artistic handwriting
```

```
CATEGORY_I CATEGORY_NAME
-----
DESCRIPTION
-----
```

```
7          Prints
Printed reproductions of artworks
```

```
8          Illustrations
Creative drawings and visual illustrations
```

```

SQL> INSERT INTO ArtProduct VALUES (101, 1, 1, 'Sunset Over Hills', 4500, 3);
1 row created.
SQL> INSERT INTO ArtProduct VALUES (102, 2, 1, 'Colors of Nature', 5200, 2);
1 row created.
SQL> INSERT INTO ArtProduct VALUES (103, 3, 2, 'Pencil Portrait', 1800, 5);
1 row created.
SQL> INSERT INTO ArtProduct VALUES (104, 4, 2, 'Charcoal Dreams', 2200, 4);
1 row created.
SQL> INSERT INTO ArtProduct VALUES (105, 5, 3, 'Digital Galaxy', 3500, 6);
1 row created.
SQL> INSERT INTO ArtProduct VALUES (106, 6, 3, 'Future City', 4000, 3);
1 row created.
SQL> INSERT INTO ArtProduct VALUES (107, 7, 4, 'Morning Light', 2800, 4);
1 row created.
SQL> INSERT INTO ArtProduct VALUES (108, 8, 5, 'The Thinker Sculpture', 7500, 2);
1 row created.
SQL> INSERT INTO ArtProduct VALUES (109, 9, 9, 'Abstract Dreams', 4200, 3);
1 row created.
SQL> INSERT INTO ArtProduct VALUES (110, 10, 10, 'Handmade Craft', 2500, 7);
1 row created.

```

```

SQL> select* from artproduct;

```

```

ARTWORK_ID ARTIST_ID  CATEGORY_I
-----
ARTWORK_NAME

```

```

-----
PRICE      STOCK
-----

```

```

101          1          1
Sunset Over Hills
4500          3

```

```

102          2          1
Colors of Nature
5200          2

```

```

ARTWORK_ID ARTIST_ID  CATEGORY_I
-----
ARTWORK_NAME

```

```

-----
PRICE      STOCK
-----

```

```

103          3          2
Pencil Portrait
1800          5

```

```

104          4          2
Charcoal Dreams

```

```

ARTWORK_ID ARTIST_ID  CATEGORY_I
-----
ARTWORK_NAME

```

```

-----
PRICE      STOCK
-----

```

```

2200          4

```

```

105          5          3
Digital Galaxy

```

5. CRUD Operations

CRUD stands for **Create, Read, Update, and Delete**. In this project, the main database operations demonstrated are Insert, Update, and Delete.

Insert Record

A new artwork can be added using the INSERT INTO command.

```
INSERT INTO ArtProduct VALUES (111, 11, 6, 'Elegant Calligraphy', 3200, 4);
```

Update Price

The UPDATE command is used to modify an existing record. Here, the price of an artwork is changed.

```
UPDATE ArtProduct SET Price = 5000 WHERE Artwork_ID = 101;
```

Delete an Artwork

The DELETE command is used to remove a particular artwork record.

```
DELETE FROM ArtProduct WHERE Artwork_ID = 111;
```

The WHERE condition identifies the specific record that needs to be updated or deleted.

```
SQL> INSERT INTO ArtProduct VALUES (111, 11, 6, 'Elegant Calligraphy', 3200, 4);
1 row created.

SQL> UPDATE ArtProduct
  2 SET Price = 5000
  3 WHERE Artwork_ID = 101;
1 row updated.

SQL> DELETE FROM ArtProduct WHERE Artwork_ID = 111;
1 row deleted.
```

6. Generate Category-wise ArtProduct Reports

SQL aggregate and grouping functions are used to generate useful reports from the ArtProduct table.

Display All Artworks Under Each Category

The SELECT command with ORDER BY is used to display artworks category-wise.

```
SELECT Category_ID, Artwork_ID, Artwork_Name, Price FROM ArtProduct ORDER BY Category_ID;
```

```
SQL> SELECT Category_ID, Artwork_ID, Artwork_Name, Price
  2 FROM ArtProduct
  3 ORDER BY Category_ID;
```

CATEGORY_I ARTWORK_ID

ARTWORK_NAME

PRICE

1 101
Sunset Over Hills
5000

1 102
Colors of Nature
5200

CATEGORY_I ARTWORK_ID

ARTWORK_NAME

PRICE

10 110
Handmade Craft
2500

2 103
Pencil Portrait

CATEGORY_I ARTWORK_ID

ARTWORK_NAME

PRICE

1800
2 104
Charcoal Dreams
2200

CATEGORY_I ARTWORK_ID

ARTWORK_NAME

PRICE

Future City
4000

3 105
Digital Galaxy
3500

CATEGORY_I ARTWORK_ID

ARTWORK_NAME

PRICE

4 107
Morning Light
2800

5 108
The Thinker Sculpture
7500

CATEGORY_I ARTWORK_ID

ARTWORK_NAME

PRICE

9 109
Abstract Dreams
4200

Count Number of Artworks in Each Category

The COUNT() function is used with GROUP BY to calculate the number of artworks in each category.

```
SELECT Category_ID, COUNT(*) AS Total_Artworks FROM ArtProduct GROUP BY Category_ID ORDER BY Category_ID;
```

```
SQL> SELECT Category_ID, COUNT(*) AS Total_Artworks FROM ArtProduct GROUP BY Category_ID ORDER BY Category_ID;

CATEGORY_I TOTAL_ARTWORKS
-----
1          2
10         1
2          2
3          2
4          1
5          1
9          1

7 rows selected.
```

Find Highest-Priced Artwork in Each Category

The MAX() function is used to find the highest artwork price in every category.

```
SELECT Category_ID, MAX(Price) AS Highest_Price FROM ArtProduct GROUP BY Category_ID ORDER BY Category_ID;
```

```
SQL> SELECT Category_ID, MAX(Price) AS Highest_Price FROM ArtProduct GROUP BY Category_ID ORDER BY Category_ID;

CATEGORY_I HIGHEST_PRICE
-----
1          5200
10         2500
2          2200
3          4000
4          2800
5          7500
9          4200

7 rows selected.
```

Display Categories Having More Than 5 Artworks

The HAVING clause is used with GROUP BY to display only categories containing more than five artworks.

```
SELECT Category_ID, COUNT(*) AS Total_Artworks FROM ArtProduct GROUP BY Category_ID
HAVING COUNT(*) > 5;
```

For the current ArtHub dataset, no category contains more than five artworks, so the result is **No rows were selected.**

```
SQL> SELECT Category_ID, COUNT(*) AS Total_Artworks FROM ArtProduct GROUP BY Category_ID HAVING COUNT(*) > 5;

no rows selected
```

Find Average Artwork Price Category-wise

The AVG() function calculates the average price of artworks in each category.

```
SELECT Category_ID, AVG(Price) AS Average_Price FROM ArtProduct GROUP BY Category_ID ORDER BY Category_ID;
```

This provides the average artwork price for each category.

```
SQL> SELECT Category_ID, AVG(Price) AS Average_Price FROM ArtProduct GROUP BY Category_ID ORDER BY Category_ID;
```

CATEGORY_I	AVERAGE_PRICE
1	5100
10	2500
2	2000
3	3750
4	2800
5	7500
9	4200

7 rows selected.